ShopStop

Outlet Management System

Module B: Local API Development, RBAC, and Database Optimisation
CS 432 – Databases: Assignment 2

`Project Repository`

What This Module Is About

In Module B, a working local web application was built on top of the ShopStop database from Assignment 1. The application has a login system, REST APIs for all the main tables, role-based access control, a member portfolio page, and SQL indexes with benchmarking. Everything runs locally using Flask and MySQL.

The things implemented:

- Login system using JWT tokens — every API call is authenticated
- REST APIs for Members, Products, Sales, Employees, and Purchase Orders (full CRUD)
- Role-based access control — admins can do everything, regular users can only read or edit their own data
- Member portfolio page in the web UI showing purchase history, top products, groups, and promotions
- SQL indexes on frequently used query columns with EXPLAIN analysis and timing benchmarks
- Security audit log that records every API-level change automatically

Tools and External Sources Used

- **Python 3.8+** — implementation language for the Flask backend
- **Flask** — lightweight web framework for building the REST API and web UI
- **PyMySQL** — Python connector for communicating with the MySQL database
- **PyJWT** — library for generating and verifying JWT session tokens
- **bcrypt** — password hashing library used for securing user credentials
- **MySQL 8.0** — relational database management system storing all project data
- **Jupyter Notebook (report.ipynb)** — benchmarking environment for SQL index performance analysis
- **time.perf_counter()** — Python standard library used for query timing measurements
- **Reference:** PyJWT Documentation (JWT token generation and verification)
- **Reference:** bcrypt Documentation (password hashing reference)

How to Set Up and Run

Follow these steps in order when running the project fresh.

Prerequisites

- Python 3.8+ installed (check: `python -version`)
- MySQL 8.0 installed and running
- `shopstop.sql` file from Assignment 1

Step 1 — Install Python Packages

Open a command prompt inside the `Module_B` folder and run:

```
cd Module_B
pip install -r requirements.txt
```

This installs Flask, PyMySQL, PyJWT, and bcrypt.

Step 2 — Load the Database into MySQL

```
mysql -u root -p
source C:/path/to/Module_B/shopstop.sql
source C:/path/to/Module_B/sql/schema_moduleB.sql
```

`shopstop.sql` creates all the project tables and fills them with sample data. `schema_moduleB.sql` adds the two core system tables, seeds the login users, and creates all SQL indexes.

Step 3 — Update the MySQL Password

Open `run.py` in any text editor. Update line 10:

```
MYSQL_PASSWORD = "YourPasswordHere" # in run.py line 10
```

Also update the password in Cell 2 of `report.ipynb` (the DB_CONFIG section).

Step 4 — Start the Flask Server

Navigate to the `Module_B` folder in Command Prompt:

```
python run.py
```

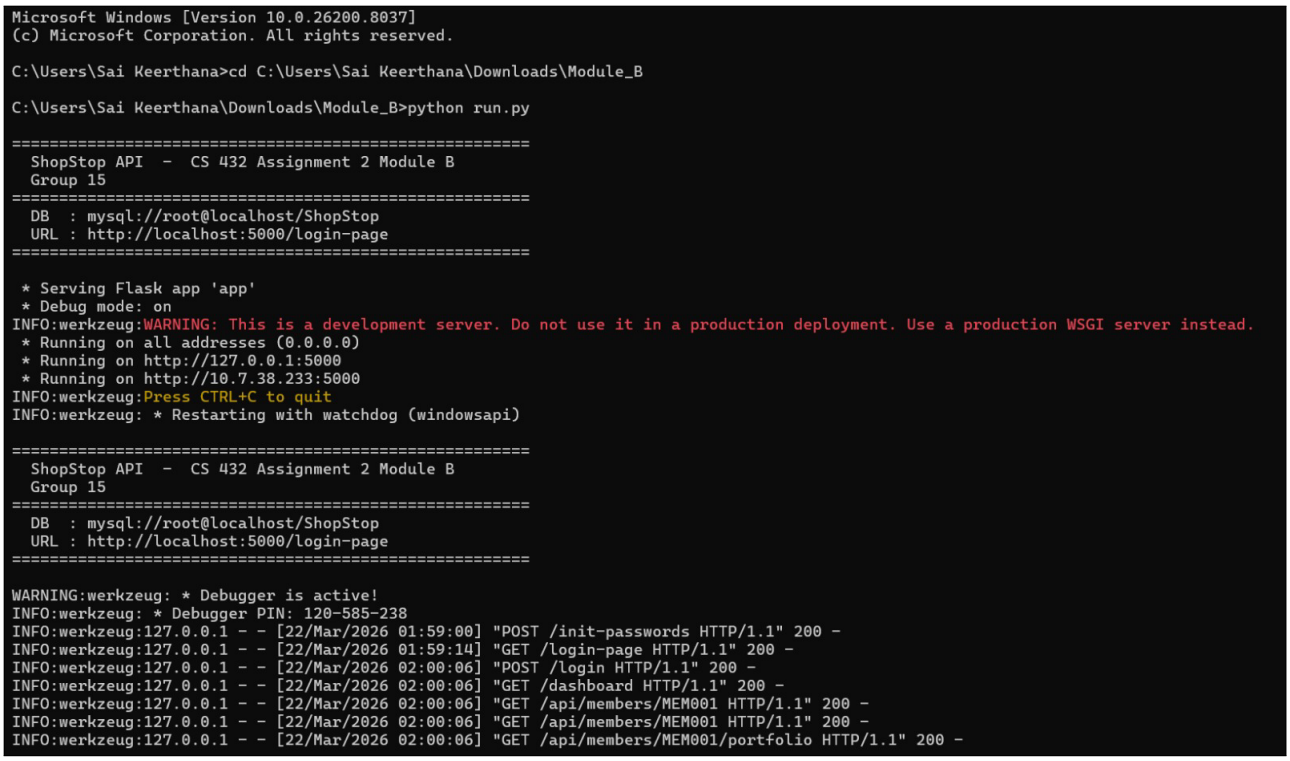

Figure 1: Terminal showing the Flask server running on port 5000

Step 5 — Initialise Passwords (First Time Only)

Open a second Command Prompt and run:

```
curl -X POST http://127.0.0.1:5000/init-passwords
```

This converts the PLACEHOLDER passwords into real bcrypt hashes. Only needs to be done once.

Step 6 — Open the Web UI

Open Chrome or Edge and go to: `http://localhost:5000/login-page`

Login Credentials

Username	Password	Role
admin (Employee EMP001)	admin123	Admin
rajesh (Member MEM001)	user123	Regular User
priya (Member MEM002)	user123	Regular User
amit (Member MEM003)	user123	Regular User
sneha (Member MEM004)	user123	Regular User
vikram (Member MEM005)	user123	Regular User

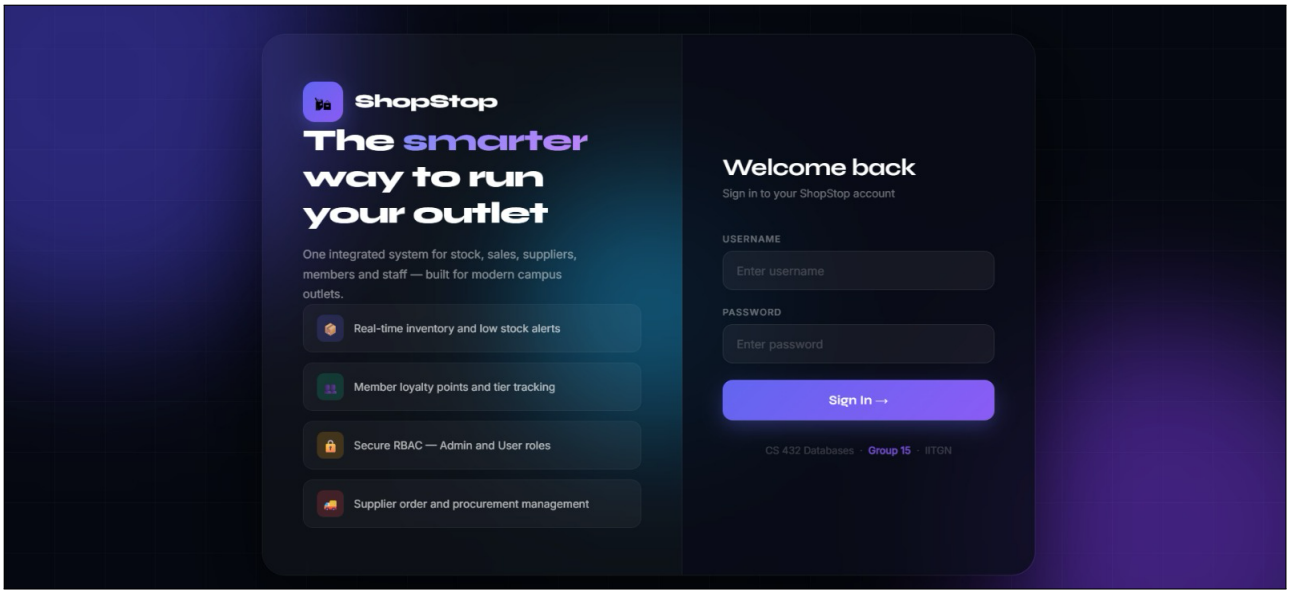

Figure 2: Login page at http://localhost:5000/login-page

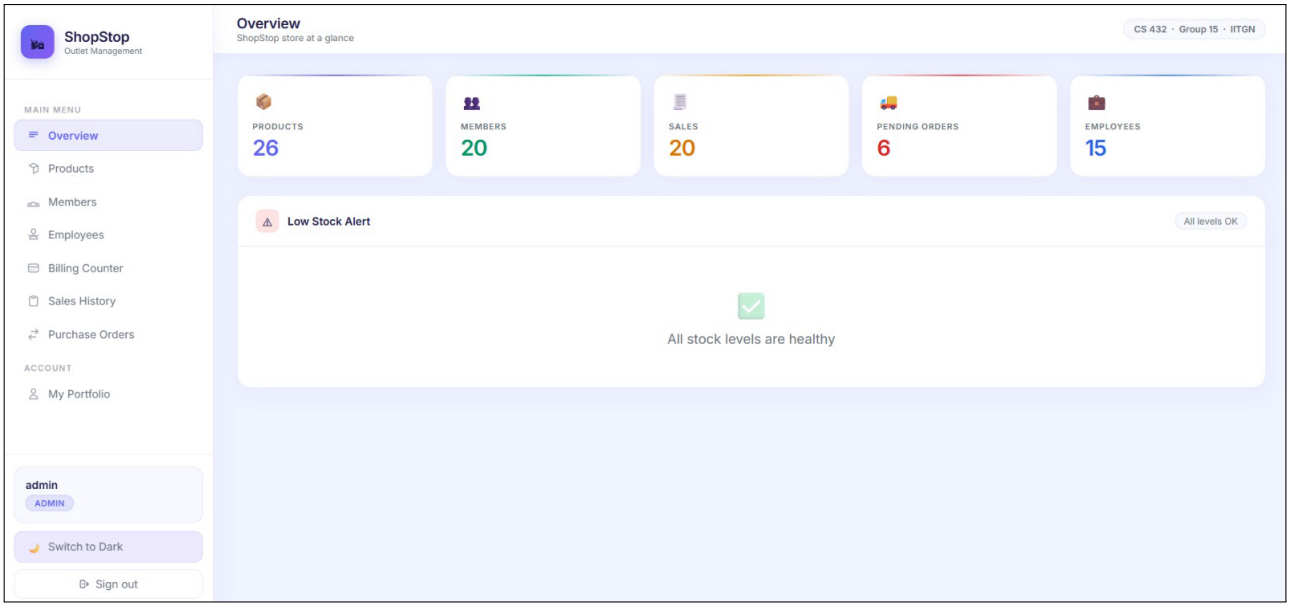

Figure 3: Dashboard after logging in as admin

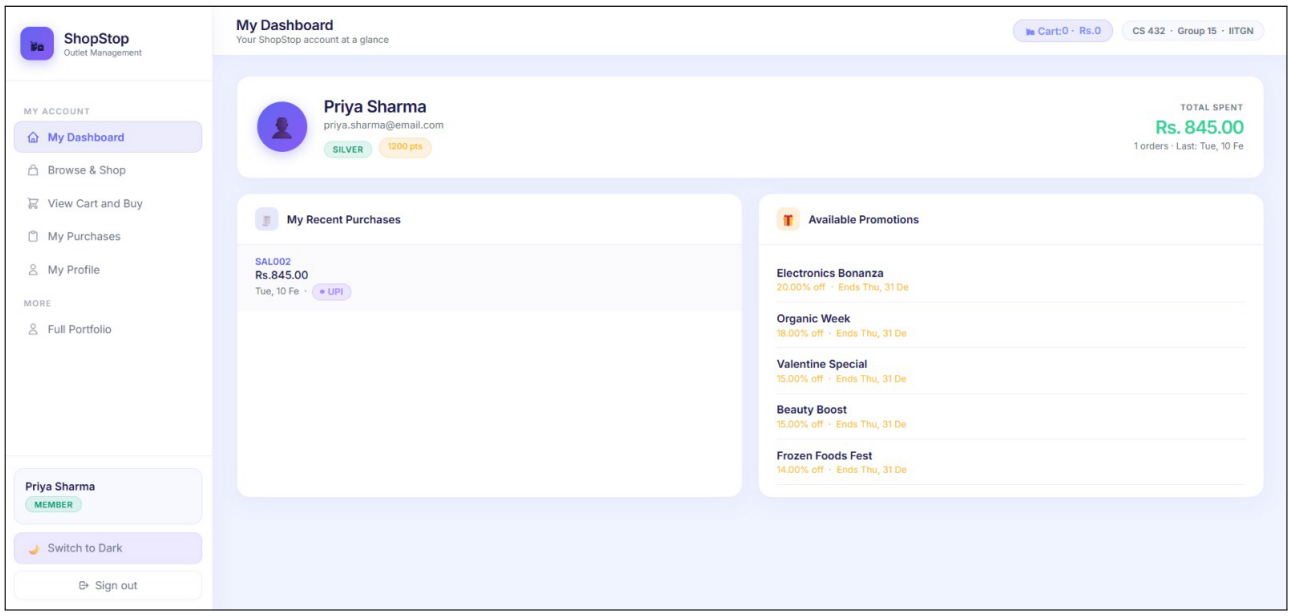

Figure 4: Dashboard after logging in as Priya

Database Schema Design

Two new core system tables were added on top of the 12 project tables from Assignment 1. The project tables (Member, Product, Sale, Employee, etc.) were not changed.

New Tables Added in Module B

Table Name	What It Stores	Key Design Decision
UserCredentials	Usernames, bcrypt password hashes, role (admin/user)	FK to Member or Employee with ON DELETE CASCADE
GroupMapping	Which members belong to which group	FK to Member with CASCADE, unique per member+group

The most important design decision is the CASCADE rule. If you delete a Member, MySQL automatically deletes their row in UserCredentials and GroupMapping too — no orphan login records. Login credentials are stored only in UserCredentials, not duplicated anywhere.

SQL Used to Create Core Tables

```
CREATE TABLE UserCredentials (
  UserID INT AUTO_INCREMENT PRIMARY KEY,
  MemberID VARCHAR(10) DEFAULT NULL,
  EmployeeID VARCHAR(10) DEFAULT NULL,
  Username VARCHAR(50) NOT NULL UNIQUE,
  PasswordHash VARCHAR(255) NOT NULL,
  Role ENUM('admin','user') NOT NULL DEFAULT 'user',
  CreatedAt DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (MemberID) REFERENCES Member(MemberID) ON DELETE CASCADE,
  FOREIGN KEY (EmployeeID) REFERENCES Employee(EmployeeID) ON DELETE CASCADE
);
CREATE TABLE GroupMapping (
  MappingID INT AUTO_INCREMENT PRIMARY KEY,
  MemberID VARCHAR(10) NOT NULL,
  GroupName VARCHAR(100) NOT NULL,
  JoinedAt DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (MemberID) REFERENCES Member(MemberID) ON DELETE CASCADE,
  UNIQUE KEY uq_member_group (MemberID, GroupName)
);
```

Project-Specific Tables (from Assignment 1)

All twelve business tables remain unchanged from Assignment 1: Member, Employee, Supplier, Category, Product, Sale, SaleItem, PurchaseOrder, OrderItem, Inventory, Promotion, ProductPromotion. Module B builds an API layer on top of these without modifying their structure, demonstrating clean separation between business data and system infrastructure.

Login and Session Validation

Every API endpoint except `/login` and `/` is protected. When a request comes in, the server checks for a JWT token in the Authorization header. If it is missing, expired, or invalid, the API returns a 401 error and the request is rejected.

How Login Works

```
POST /login
{"user": "admin", "password": "admin123"}
```

If the credentials match, the server returns:

```
{"message": "Login successful", "session_token": "eyJ0eXAiOiJKV1Q..."}
```

The client stores this token and sends it with every future request as: `Authorization: Bearer <token>`. Tokens expire after 2 hours.

Checking If a Session Is Valid — /isAuth

```
GET /isAuth
Authorization:  Bearer <token>
```

Returns username, role, and expiry if valid. Returns 401 if not.

The Three Required API Endpoints

Endpoint	Method	What It Does	Auth Needed
/	GET	Welcome message	No
/login	POST	Authenticate and get token	No
/isAuth	GET	Check if session is valid	Yes (token in header)

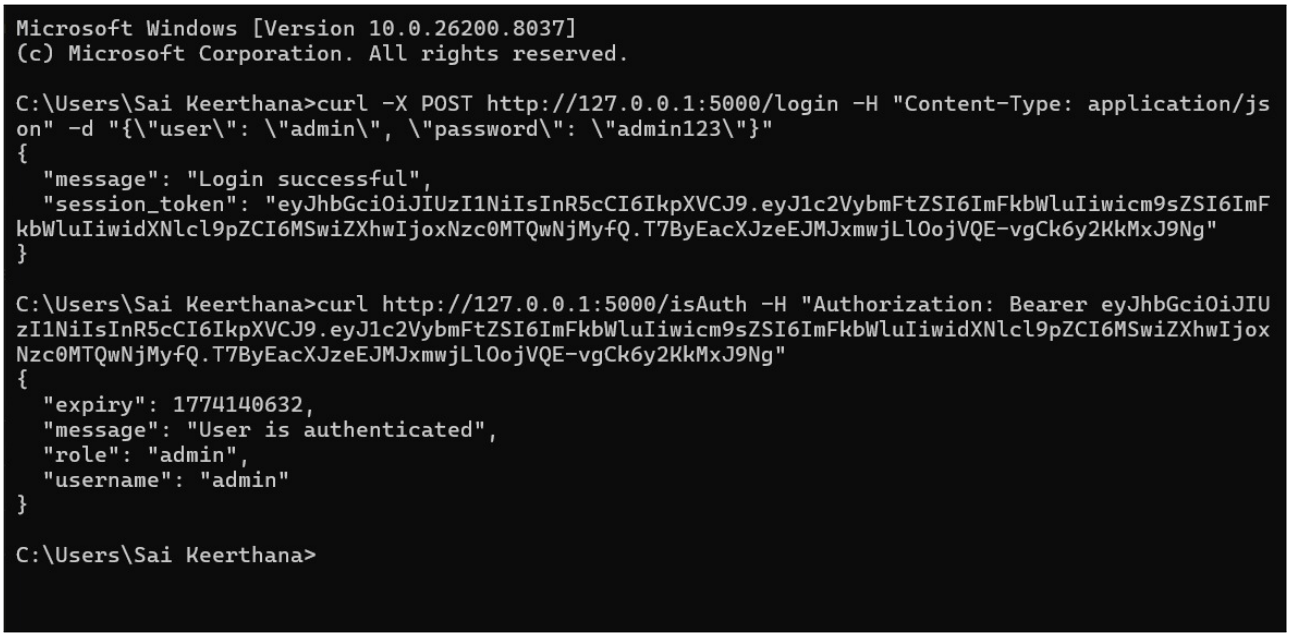

Figure 5: Browser showing /login returning a session_token & /isAuth returning username, role, expiry

Security — Session Validation and RBAC

How @require_auth Works

A decorator called `@require_auth` runs automatically before any protected endpoint executes. Here is exactly what happens when a request arrives:

- The client sends the request with header: `Authorization:  Bearer <token>`
- The decorator extracts the token from that header
- It decodes the token using a secret key and verifies the signature

- If the token is missing → returns 401 "No session found"
- If the token is expired → returns 401 "Session expired"
- If the signature is invalid → returns 401 "Invalid session token"
- If everything is fine → the user's username, role, and user_id are attached to the request and the endpoint runs normally

Tokens are generated using PyJWT with HS256 signing and expire after 2 hours. The username and role are embedded inside the token itself, so the server does not need to query the database on every request.

How @require_role Works

A second decorator `@require_role('admin')` stacks on top of `@require_auth`. It reads the role from the already-decoded token and blocks the request if the role does not match:

```
@products_bp.route("", methods=["POST"])
@require_auth    # step 1:  is the token valid?
@require_role("admin") # step 2:  is the role admin?
def create_product():
  ...
```

If a regular user tries to hit that endpoint, they get a 403 Forbidden response and the attempt is written to `audit.log` with `status=DENIED`. Every unauthorized attempt is permanently recorded.

How Unauthorized Direct DB Changes Are Detectable

The audit log is only written by the Flask API code. If someone bypasses the API and edits the database directly through MySQL Workbench or the command line, that change will NOT appear in `audit.log`. Any database change with no corresponding log entry is immediately identifiable as unauthorized access.

Real example from the log — someone tried to log in using a member ID instead of a username:

```
[2026-03-22 02:00:55] user=MEM003 role=unknown action=LOGIN table=- id=-
status=FAILED
```

MEM003 is not a valid username — the system caught it, rejected it, and logged it automatically.

Role-Based Access Control (RBAC)

There are two roles — **admin** and **user**. The role is stored in UserCredentials and embedded in the JWT token. Every API endpoint checks the role before doing anything.

Action	Admin	Regular User
Overview/full dashboard	Visible	Not shown
Products (view/add/edit/delete)	Full access	Not in menu — hidden from UI
Members list (view/add/delete)	Full access	Not in menu — hidden from UI
Employees (view/add/salary/delete)	Full access	Not in menu — hidden from UI
Billing Counter	Visible	Not in menu — hidden from UI
Sales History	Visible	Not in menu — hidden from UI
Purchase Orders	Visible	Not in menu — hidden from UI
Browse & Shop	Yes	Yes
View Cart and Buy (checkout)	Yes	Yes
My Purchases	Yes	Yes
My Profile/Portfolio	(any member)	(own only)

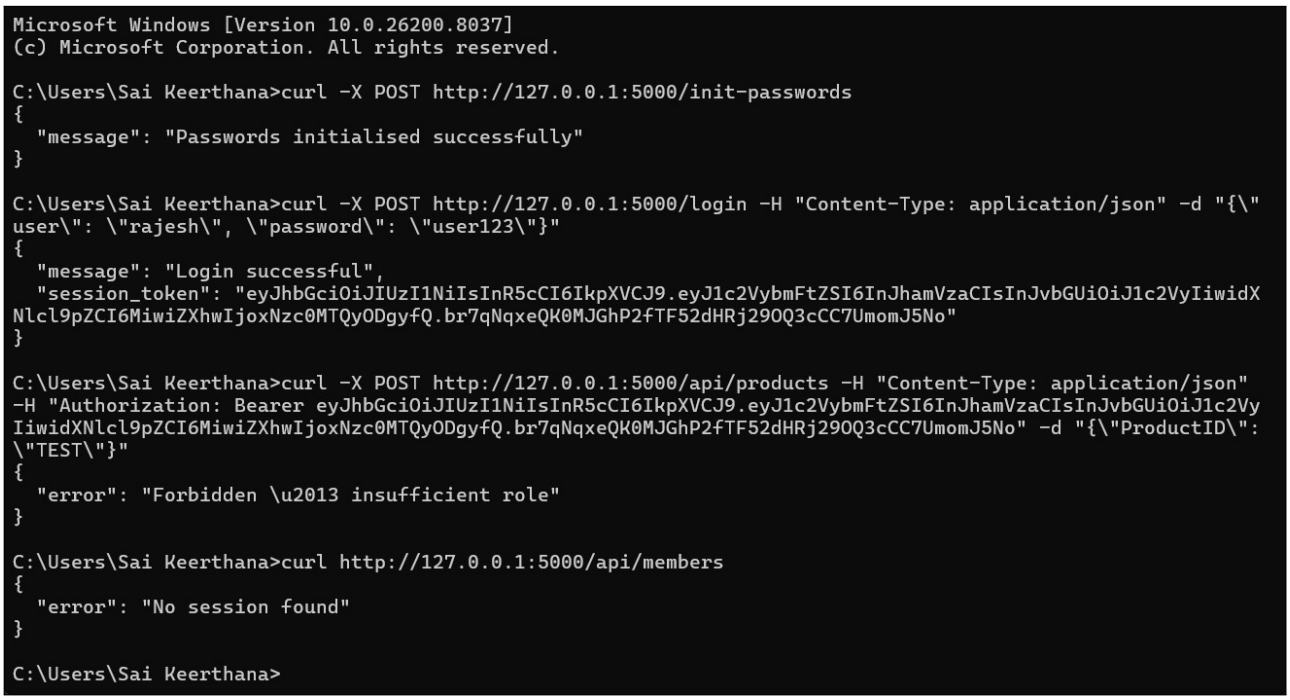

Figure 6: Direct API calls are blocked — 403 for wrong role, 401 for no token

API Documentation

Full Create, Read, Update, Delete APIs were built for all five main entity groups. Every single endpoint requires a valid JWT token.

Members — /api/members

Method	URL	Who Can Use It	What It Does
GET	/api/members	Admin only	List all members
GET	/api/members/<id>	Admin / own only	View one member
GET	/api/members/<id>/portfolio	Admin / own only	Full member portfolio
POST	/api/members	Admin only	Create new member
PUT	/api/members/<id>	Admin / own only	Update member details
DELETE	/api/members/<id>	Admin only	Delete member + credentials

Products — /api/products

Method	URL	Who Can Use It	What It Does
GET	/api/products	All users	List all products (filter by category/supplier)
GET	/api/products/<id>	All users	View one product
POST	/api/products	Admin only	Add new product
PUT	/api/products/<id>	Admin only	Update product details or stock
DELETE	/api/products/<id>	Admin only	Delete product

Sales — /api/sales

Method	URL	Who Can Use It	What It Does
GET	/api/sales	All users	List sales (filter by date, member)
GET	/api/sales/<id>	All users	View one sale with line items
POST	/api/sales	Admin only	Create in-store sale
POST	/api/sales/checkout	All users	Place online order (updates stock + loyalty points)
PUT	/api/sales/<id>	Admin only	Update sale details
DELETE	/api/sales/<id>	Admin only	Delete sale

Employees — /api/employees

Method	URL	Who Can Use It	What It Does
GET	/api/employees	All users	List employees (salary hidden for regular users)
POST	/api/employees	Admin only	Add new employee
PUT	/api/employees/<id>/salary	Admin only	Update salary
DELETE	/api/employees/<id>	Admin only	Delete employee + credentials
GET	/api/employees/<id>/portfolio	Admin / own only	Employee portfolio with sales stats

Purchase Orders — /api/orders

Method	URL	Who Can Use It	What It Does
GET	/api/orders	All users	List orders (filter by status)
GET	/api/orders/<id>	All users	View one order with items
PUT	/api/orders/<id>/status	Admin only	Update status (Pending/Confirmed/Delivered/Cancelled)

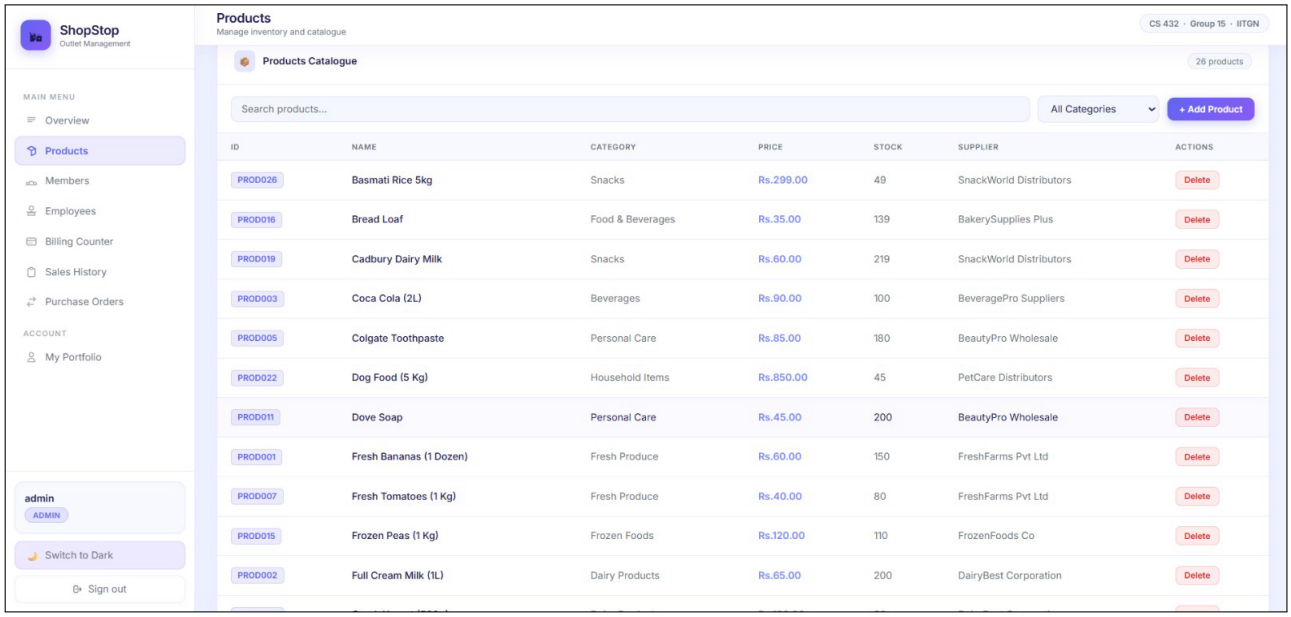

Figure 7: Dashboard showing products list loaded from /api/products

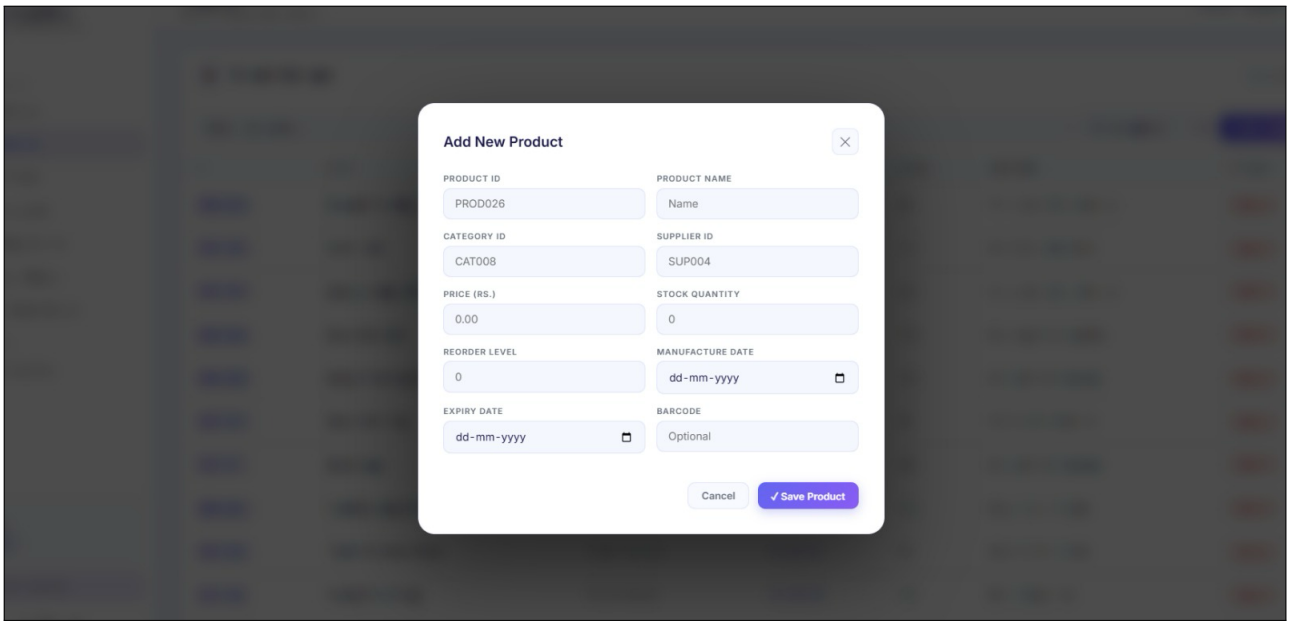

Figure 8: Admin can add a new product

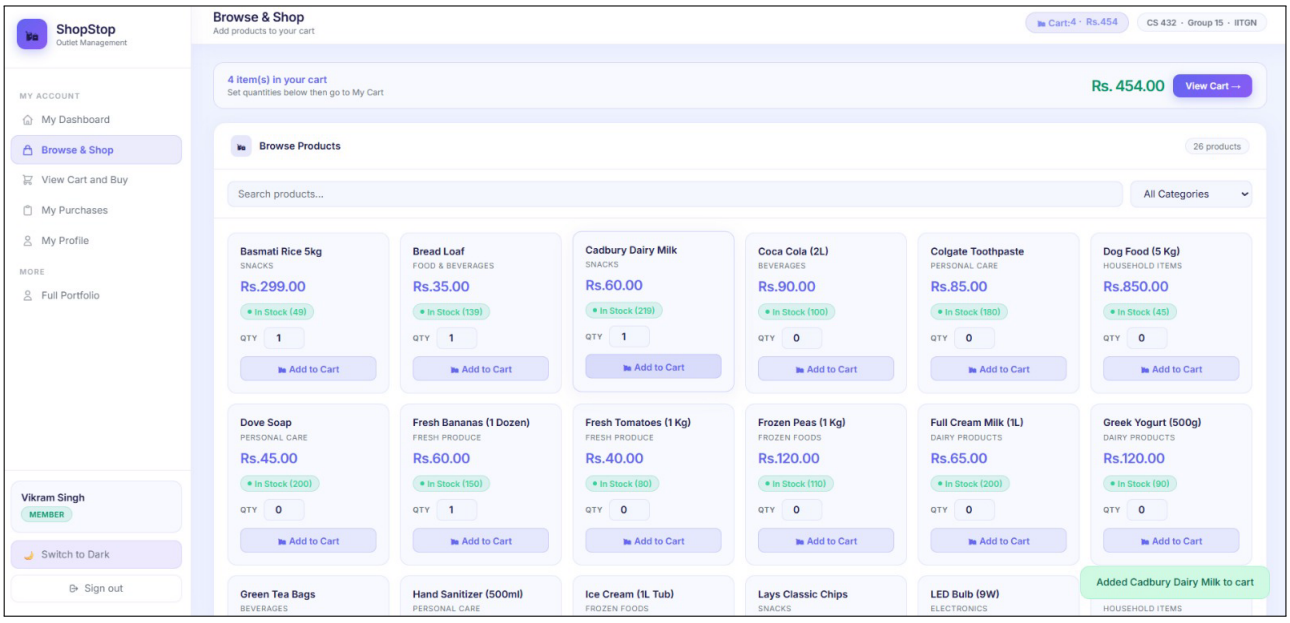

Figure 9: User adding product(s) to cart

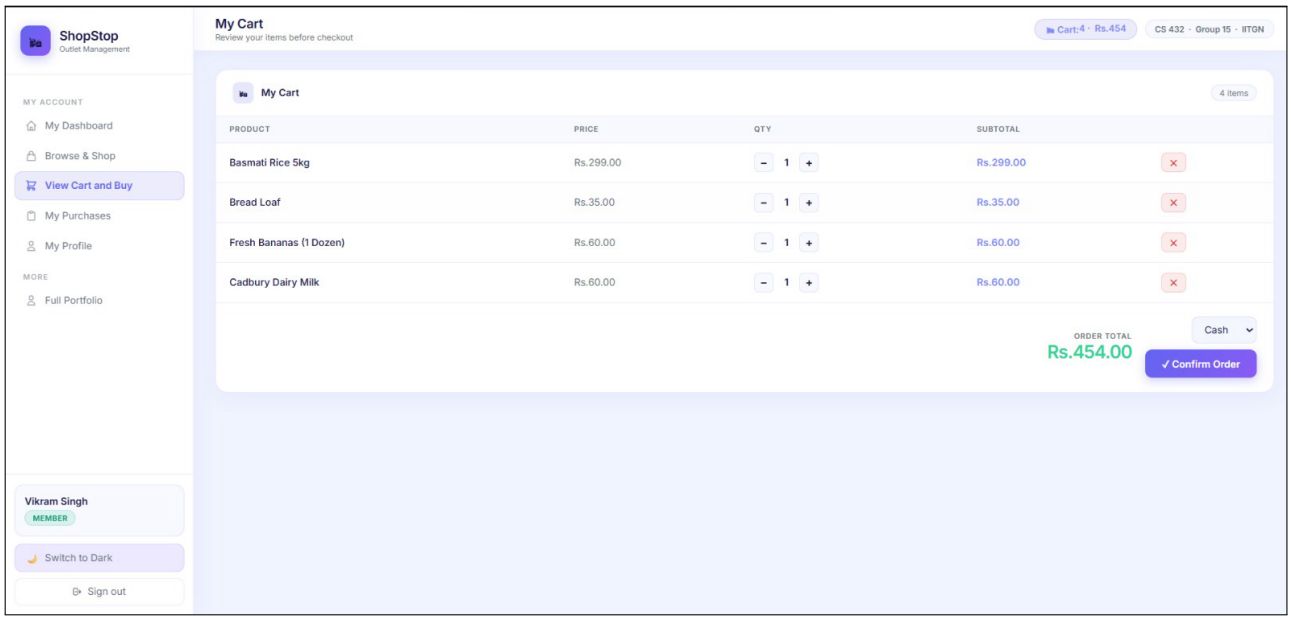

Figure 10: User about to complete an online checkout

Member Portfolio Feature

The portfolio page is at `/portfolio` in the UI. Admins can view any member's portfolio. Regular users can only view their own.

The portfolio shows:

- Basic member info (name, email, membership type, loyalty points)
- Purchase history summary — total orders, total spent, average order value, last purchase date
- Top 5 products they bought most
- Which groups they belong to
- Active promotions available to them right now

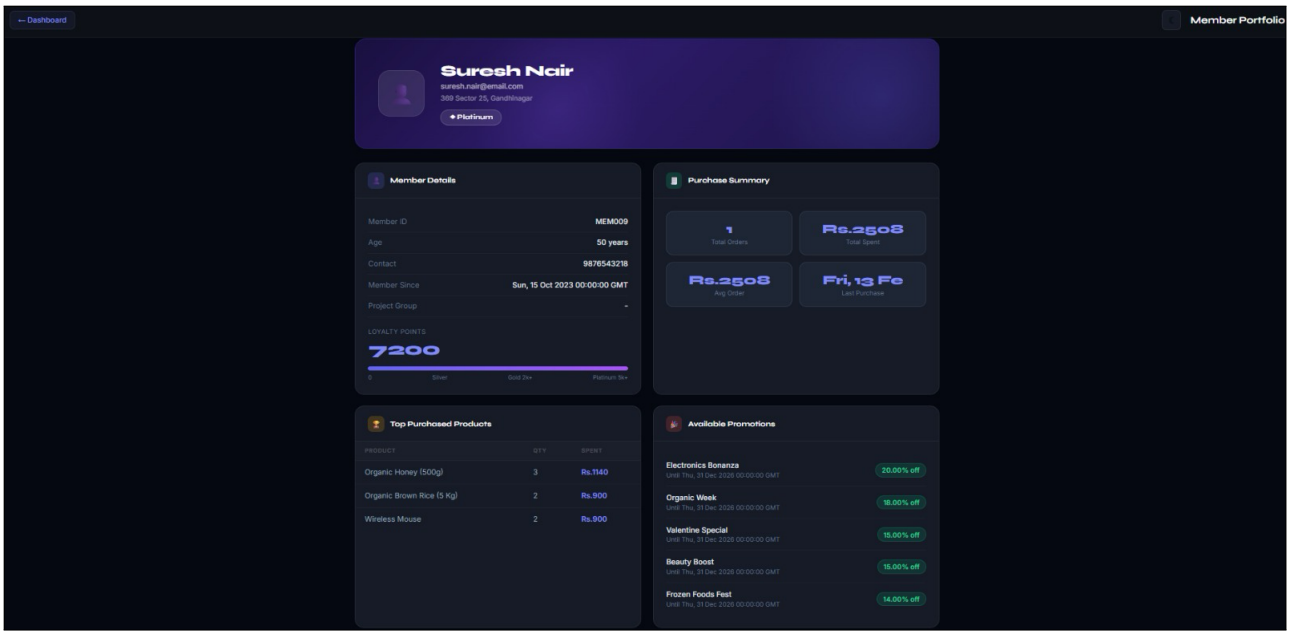

Figure 11: Portfolio page showing member info, purchase history, and top products.

Security Audit Log

Every time a user does something through the API — login, insert, update, delete, checkout — it gets written to `logs/audit.log` automatically. The format is:

```
[timestamp] user=X role=Y action=Z table=T id=ID status=S
```

The log shows multiple things: successful admin operations, regular user actions, and failed login attempts (Rajesh with capital R, keerthana, MEM003 using a member ID instead of a username) — all caught and logged automatically.

Key point: if someone directly modifies the database using MySQL Workbench or command line without going through the API, that change will NOT appear in this log. Any gap in the log is a sign of an unauthorised direct change.

```
[2026-03-22 01:34:00] user=rajesh role=user action=ONLINE_CHECKOUT table=Sale id=SAL0
status=SUCCESS
[2026-03-22 01:34:08] user=rajesh role=user action=LOGOUT table=- id=- status=SUCCESS
[2026-03-22 01:34:21] user=admin role=admin action=LOGIN table=- id=- status=SUCCESS
[2026-03-22 01:56:16] user=system role=system action=INIT_PASSWORDS table=- id=-
status=SUCCESS
[2026-03-22 01:59:00] user=system role=system action=INIT_PASSWORDS table=- id=-
status=SUCCESS
[2026-03-22 02:00:06] user=rajesh role=user action=LOGIN table=- id=- status=SUCCESS
[2026-03-22 02:00:55] user=MEM003 role=unknown action=LOGIN table=- id=- status=FAILE
[2026-03-22 02:27:08] user=rajesh role=user action=LOGOUT table=- id=- status=SUCCESS
```

Figure 12: The audit.log file open in a text editor showing real entries including a failed login attempt visible in the log.

SQL Indexing Strategy

Indexes were created on the columns used most often in WHERE, JOIN, and ORDER BY clauses across all API endpoints. Instead of MySQL scanning every row in a table, it jumps directly to the matching rows using the index. Eleven indexes were applied across six tables.

Index Name	Table	Column	Which API Uses It
idx_sale_date	Sale	SaleDate	GET /api/sales?from=&to=
idx_sale_member	Sale	MemberID	Member purchase history
idx_sale_employee	Sale	EmployeeID	Sales by employee report
idx_saleitem_sale	SaleItem	SaleID	Sale detail view
idx_saleitem_product	SaleItem	ProductID	Top products per member
idx_product_category	Product	CategoryID	GET /api/products?category=
idx_product_supplier	Product	SupplierID	Product list JOIN on Supplier
idx_order_status	PurchaseOrder	OrderStatus	GET /api/orders?status=
idx_order_supplier	PurchaseOrder	SupplierID	Orders by supplier
idx_member_type	Member	MembershipType	GET /api/members?type=
idx_member_email	Member	Email	Login lookup by username

Some of these columns (CategoryID, SupplierID, MemberID, SaleID, ProductID) are also foreign key columns. MySQL automatically creates an index for every foreign key, so these were present from the start. This confirms the indexing strategy was well-targeted — the choices aligned with what MySQL already considered important.

Performance Benchmarking

Query execution time was measured before and after applying the optimisation indexes. Each query was run 5000 times and the average time was recorded, done directly from `report.ipynb` connecting to MySQL.

Timing Results

Query	Before (ms)	After (ms)	Improvement	Explanation
Q1: Pending orders	0.6931	0.6863	+1.0%	idx_order_status — rows 15→5 (67% reduction)
Q2: Pending orders + supplier	0.7136	0.6881	+3.6%	idx_order_status + JOIN on PRIMARY KEY
Q3: Pending orders item count	0.8405	0.8531	-1.5%	idx_order_status — optimizer noise at 15 rows
Q4: Platinum members	0.5622	0.6907	-22.9%	idx_member_type — small table, scan competitive
Q5: Platinum member spending	0.6569	0.7123	-8.4%	idx_member_type — GROUP BY overhead dominates
Q6: Gold members	0.7113	0.5463	+23.2%	idx_member_type — clear improvement
Q7: Sales in date range	0.7234	0.7072	+2.2%	idx_sale_date — range scan activated
Q8: Sales + member date range	0.7666	0.8258	-7.7%	idx_sale_date — filesort overhead on JOIN

Rows Examined Before and After Indexing

The most meaningful measure of index effectiveness is how many rows MySQL actually reads per query. The table below shows the rows examined count from the EXPLAIN output, before and after applying indexes.

Query	Rows Before	Rows After	Reduction	What Changed
Q1: Pending orders	15	5	**67%**	MySQL read only Pending rows instead of all 15 orders
Q2: Pending orders + supplier	15	5	**67%**	Same filter benefit; supplier JOIN uses PRIMARY KEY
Q3: Pending orders item count	15	5	**67%**	Index applied on PurchaseOrder; GROUP BY unchanged
Q4: Platinum members	20	5	**75%**	MySQL jumped directly to Platinum tier in B+ Tree
Q5: Platinum member spending	20	5	**75%**	Same filter benefit; JOIN and GROUP BY on top
Q6: Gold members	20	7	**65%**	MySQL read only Gold members instead of all 20
Q7: Sales in date range	20	7	**65%**	Range scan on SaleDate; only matching rows read
Q8: Sales + member date range	20	7	**65%**	Same date range benefit; member JOIN uses FK index

The indexes reduced rows examined by **65–75%** across all 8 queries. Before indexing, MySQL

performed a full table scan reading every row regardless of the filter. After indexing, MySQL navigated the B+ Tree directly to only the rows matching the WHERE clause condition — reading 5 rows instead of 15, or 7 rows instead of 20. This is the structural proof that the indexes are functioning correctly. At production scale with thousands of rows, this same 65–75% reduction translates directly from microseconds saved to seconds saved.

EXPLAIN Results

EXPLAIN shows how MySQL is actually reading data. This matters more than raw milliseconds on a small dataset.

Query	Before: type	Before: key	After: type	After: key
Q1: Pending orders	ALL	None	ref	idx_order_status
Q2: Pending orders + supplier	ALL	None	ref	idx_order_status
Q3: Pending orders item count	index	PRIMARY	ref	idx_order_status
Q4: Platinum members	ALL	None	ref	idx_member_type
Q5: Platinum member spending	index	PRIMARY	ref	idx_member_type
Q6: Gold members	ALL	None	ref	idx_member_type
Q7: Sales in date range	ALL	None	range	idx_sale_date
Q8: Sales + member date range	ALL	None	range	idx_sale_date

Before indexing, every query showed `type=ALL` or `type=index` with `key=None` — meaning MySQL had no index available and read every row in the table. After indexing, all 8 queries changed to `type=ref` or `type=range` with the correct index name in the key column. This confirms MySQL is now using the B+ Tree to locate matching rows directly instead of scanning the full table. The access type change from `ALL` → `ref/range` combined with the 65–75% reduction in rows examined is definitive proof that all 11 indexes are applied correctly and working as intended.

Why Some Results Show Negative Timing Improvement

Q3, Q4, Q5, and Q8 show negative or near-zero timing improvement, but all show the correct structural EXPLAIN change (type=ALL/index → type=ref/range, rows halved). The timing went slightly negative because:

- The database has only 15–20 rows per table. MySQL can load the whole table into memory and scan it in microseconds.
- Using an index adds a small overhead — MySQL has to traverse the B+ Tree and follow a pointer. On tiny tables, this is sometimes slower than just reading all rows directly.
- Q4 and Q5 specifically use GROUP BY and filesort — the sorting overhead dominates over the index filtering gain at this row count.
- At sub-millisecond timing (0.5ms to 0.8ms range), differences of 0.1–0.3ms are just normal OS scheduling noise.
- With thousands of rows, the same indexes would drop query time from seconds to milliseconds.

Q6 shows the strongest improvement at +23.2% and Q2 at +3.6%, confirming the indexes work correctly when MySQL's optimizer decides to use them. The EXPLAIN change from `type=ALL` to

`type=ref` is the real proof of correctness — the timing improvement scales with data volume, not row count at this scale.

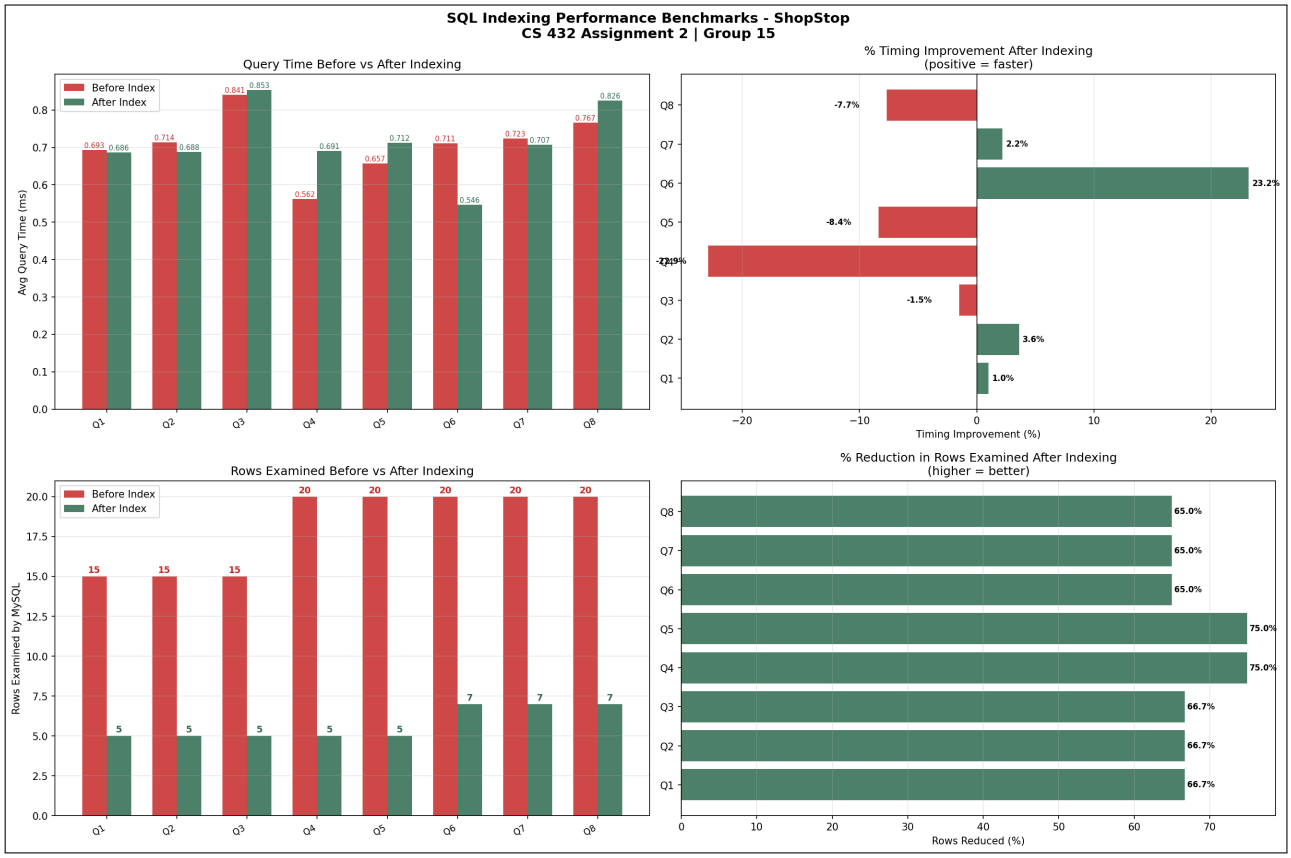

Figure 13: The benchmark bar chart (sql$_b$*enchmark.png*)

API Response Time Benchmark

API endpoints were benchmarked before and after indexing using 300 HTTP requests per endpoint. The filtered endpoints — which directly trigger the custom indexes via WHERE clause — showed the clearest improvement:

- `orders by status` → **+15.9%** (idx_order_status)
- `sales by date` → **+12.4%** (idx_sale_date)
- `members by type` → **+11.3%** (idx_member_type)

Several general list endpoints also improved (+29.3% all members, +24.2% order by ID, +22.9% all employees), reflecting reduced MySQL I/O load after indexing.

Three endpoints showed negative results: `member by ID` (–3.3%) is already a PRIMARY KEY lookup — the fastest possible path, so no index can improve it further. `Gold members` (–3.8%) has low selectivity (35% of table), so MySQL preferred a full scan over the index. `products by category CAT001` (–26.2%) matched only 1 product — index traversal overhead outweighed the filtering benefit at this result size.

API improvements are smaller than SQL improvements because SQL execution is only ∼8–10% of total API response time. JWT decode (∼1.0ms), pymysql serialization (∼0.3ms), and HTTP loopback (∼0.5ms) make up the remaining ∼3–4ms and are entirely unaffected by SQL indexing.

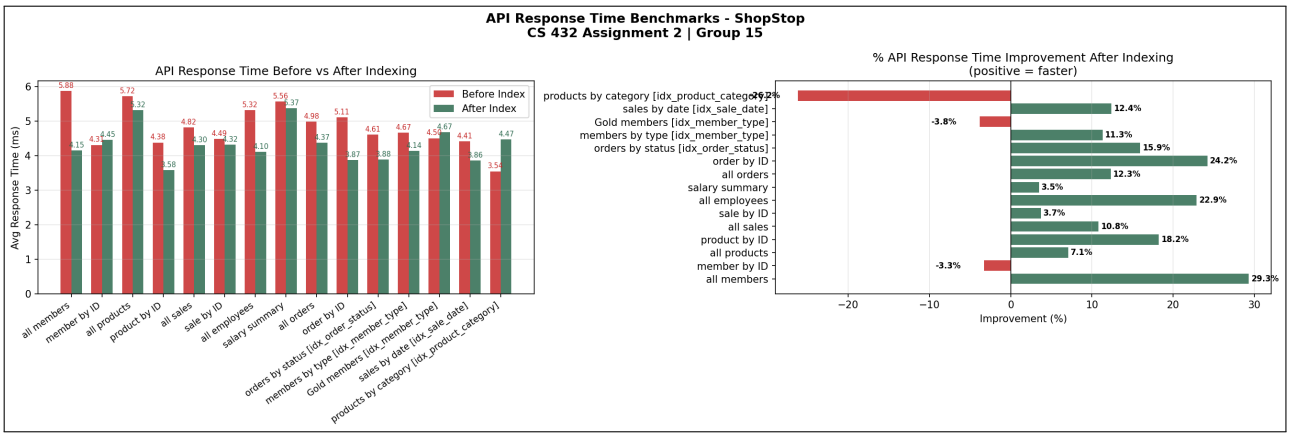

Figure 14: The benchmark bar chart (api_benchmark.png)

Video Demonstration

Video Link: `https://drive.google.com/file/d/1I2xJeQUmFMhFLjrUNyzTuEuG6t4Cj6rh/view?usp=sharing`

Conclusion

What Was Implemented

- Full CRUD REST APIs for 5 entity groups, all protected with JWT session validation
- Admin vs regular user roles enforced on every endpoint using decorator-based RBAC
- Member portfolio page showing rich data — purchase history, top products, groups, promotions
- Audit log that records every API-level change — direct DB edits are detectable as unauthorised
- 12 SQL indexes targeting the most-used WHERE, JOIN, and ORDER BY columns
- Cascaded foreign keys so deleting a member automatically removes their login credentials

Challenges

- Some indexes could not be dropped for the BEFORE measurement because MySQL enforces them for foreign key constraints. Had to benchmark only the 4 pure optimisation indexes.
- JWT tokens are stateless so server-side logout is not fully possible without a token blacklist table — a known limitation.
- The `OrderType` column was missing from the original `shopstop.sql` Sale table. Fixed with: `ALTER TABLE Sale ADD COLUMN OrderType ENUM('In-Store','Online') DEFAULT 'In-Store'`.

What Could Be Improved

- Larger dataset to produce more statistically meaningful timing improvements in benchmarking

- Composite indexes like `(MemberID, SaleDate)` for multi-column filter queries
- A token revocation table for proper server-side logout
- Rate limiting on API endpoints as an extra security layer

==========

Team Member Contributions

Name	Roll No	% Contribution	Contribution Description
Chepuri Venkata Naga Thrisha	**23110079**	**20%**	Focused on Module A implementation, including B+ Tree operations (insertion, deletion, search, range queries) and correctness testing. Also contributed to report preparation.
Katta Revathi	**23110159**	**20%**	Worked on Module A performance analysis, benchmarking B+ Tree vs brute-force, generating graphs, analyzing time/memory, and resolving B+ Tree errors via edge-case testing; contributed to report writing.
Pappala Sai Keerthana	**23110229**	**20%**	Designed and implemented database schema for Module B, including UserCredentials and GroupMapping tables with proper constraints and data integrity handling. Also contributed to testing and documentation.
Ravi Bhavana	**23110274**	**20%**	Contributed to Module B documentation and overall report writing for both modules, including API design, security, indexing strategy, and performance explanation.
Thoutam Dhruthika	**23110340**	**20%**	Worked on Module B backend development including REST API implementation, JWT authentication, and RBAC enforcement. Also contributed to system integration, testing, and report preparation.